

一、胜利检测（Victory Detection）：给 Agent 装上“眼睛”

目标：让系统知道什么时候该停手。在 ARC-AGI-3 中，胜利通常是“输出网格完全等于目标网格”。

植入位置：Layer 0 (感知) 或 Layer 1 (世界模型)。

具体做法：

1. **数据比对：**在每一步动作执行后，将当前的输出网格（Prediction）与目标网格（Target）进行像素级的对比。
2. **触发信号：**一旦完全一致，设置一个标志位，并让 `is_done` 返回 `True`。

代码实现（参考）：

```
python
```

在 `WorldModelField` 或单独的检测模块中

```
class VictoryDetector:
    def check(self, current_grid, target_grid):
        # 简单的全网格比对
        if current_grid.shape != target_grid.shape:
            return False
        # 严格相等判定
        if np.array_equal(current_grid, target_grid):
            return True
        return False
```

在 Agent 的 `choose_action` 循环中调用

```
if self.victory_detector.check(latest_frame, target_frame):
    return None # 或者返回特定的结束动作
```

二、搜索算法（Search Algorithm）：给 Agent 装上“大脑”

目标：利用 Layer 1 构建的转移表（Transition Table），找到最高效的解题路径，避免在死胡同里浪费步数。

植入位置：Layer 3 (Planner)。

算法选择:

- **Breadth-First Search (BFS):** 适合寻找“最少步数”的解法。在 ARC 这种规则明确的游戏，BFS 往往比复杂的 RL 更有效。
- **Greedy Best-First Search:** 结合你的“信息增益”评分，优先探索能最大程度改变网格状态的动作。

具体做法:

1. **构建图 (Graph):** 利用 WorldModelField 记录的 $(s, a) \rightarrow s'$ 关系。
2. **定义代价 (Cost):** 每一步的代价是 1。如果动作导致网格回到之前的状态 (死循环)，代价设为无穷大。
3. **路径回溯:** 找到目标状态后，回溯动作序列。

代码实现 (参考):

```
python

在 LightweightPlanner 中实现

class BFSSearch:
    def search(self, start_state, goal_checker, world_model):
        queue = deque([(start_state, [])]) # (状态, 动作路径)
        visited = set()
        while queue:
            state, path = queue.popleft()

            # 检查是否胜利
            if goal_checker(state):
                return path # 返回动作列表

            # 标记为已访问
            state_hash = self.hash_state(state)
            if state_hash in visited: continue
            visited.add(state_hash)
```

```
# 探索邻居

actions = ['UP', 'DOWN', 'LEFT', 'RIGHT']

for action in actions:

    next_state = world_model.predict(state, action) # 使用世界模型预测下一步

    # 剪枝：如果预测的状态无效或已访问，跳过

    if next_state is not None:

        new_path = path + [action]

        queue.append((next_state, new_path))

return None # 找不到路
```

在 Agent 决策时调用

```
path = self.planner.search(current_state, self.victory_detector, self.field)

if path:

    next_action = path[0] # 执行第一步
```

三、 整合后的决策流程

将这两个模块整合进 `choose_action`，Agent 逻辑会变成：

1. **感知**：获取当前图像。
2. **检测**：判断是否已胜利？如果是，结束。
3. **搜索**：如果没有胜利，调用 BFS 搜索从当前状态到胜利状态的最短路径。
 - **如果搜到路**：直接执行路径上的第一个动作。
 - **如果没搜到路**（比如陷入死胡同或规则太复杂）：
 - 回退到“探索模式”：随机选择信息增益最高的动作，或者调用 LLM 寻求建议。
4. **记录**：将（状态，动作，新状态）存入转移表。